C++

C++ ESSENTIALS 1
PROJECT PORTFOLIO

Aligned with Cisco Networking Academy Curriculum

48 Hands-On Projects | 8 Course Modules | 3 Difficulty Tiers

Basic • Intermediate • Advanced — 2 Projects per Level per Module

Intro & Structure	**Types & Operators**	**Conditionals**	**Loops**
Arrays & Vectors	**Functions**	**Pointers & Memory**	**Strings**

CPA: C++ Certified Associate Programmer Preparation | August 2026

TABLE OF CONTENTS

Module 1: Introduction to C++ & Program Structure

- [Basic] Personal Profile Card Generator
- [Basic] Interactive Greeting Machine
- [Intermediate] ASCII Art Scene Renderer
- [Intermediate] Multi-File Hello World with Makefile
- [Advanced] Build System Simulator
- [Advanced] Interactive C++ Syntax Reference Tool

Module 2: Data Types, Variables & Operators

- [Basic] Universal Unit Converter
- [Basic] Character Code Explorer
- [Intermediate] Bitwise Operations Visualizer
- [Intermediate] Financial Compound Interest Calculator
- [Advanced] Data Type Limits & Overflow Detector
- [Advanced] Expression Parser & Precedence Evaluator

Module 3: Conditional Statements

- [Basic] Student Grade Classification System
- [Basic] Day Planner Decision Engine
- [Intermediate] Scientific Calculator with Switch-Case
- [Intermediate] Leap Year & Calendar Validator
- [Advanced] Text-Based Adventure Game Engine
- [Advanced] Income Tax Computation Engine

Module 4: Loops & Iteration

- [Basic] Multiplication Table Generator
- [Basic] Number Guessing Game
- [Intermediate] Pattern Printing Workshop
- [Intermediate] Prime Number Toolkit
- [Advanced] Console-Based Snake Game (Simplified)
- [Advanced] Numerical Methods Calculator

Module 5: Arrays & Vectors

- [Basic] Student Marks Analyzer
- [Basic] Dynamic Shopping List Manager
- [Intermediate] Tic-Tac-Toe Game with 2D Array
- [Intermediate] Sorting Algorithm Visualizer
- [Advanced] Matrix Operations Library
- [Advanced] Student Database with Vector of Records

Module 6: Functions

- [Basic] Math Utilities Function Library
- [Basic] Temperature & Currency Converter Suite
- [Intermediate] Array Processing with Pass-by-Reference
- [Intermediate] Recursive Art: Fractals in Text
- [Advanced] Recursive Puzzle Solver (Towers of Hanoi & N-Queens)

- [Advanced] Mini Testing Framework

Module 7: Pointers & Dynamic Memory

- [Basic] Pointer Playground & Visualizer
- [Basic] Swap Functions: Value vs Pointer vs Reference
- [Intermediate] Dynamic Array Manager
- [Intermediate] Pointer-Based String Operations
- [Advanced] Memory Pool Allocator Simulation
- [Advanced] Linked List from Scratch with Pointers

Module 8: Strings & String Operations

- [Basic] Word Counter & Text Statistics
- [Basic] Password Strength Validator
- [Intermediate] Text Encryption/Decryption Tool
- [Intermediate] CSV Line Parser with String Streams
- [Advanced] Regex-Free Text Search Engine
- [Advanced] Markdown-to-Plain-Text Converter

CURRICULUM OVERVIEW

This guide maps directly to the Cisco Networking Academy C++ Essentials 1 (CPA) course. Each module covers a core pillar of C++ programming, progressing from basic syntax through control flow, data structures, functions, pointers, and string manipulation. The 48 projects are designed to reinforce every key concept with increasing complexity.

BASIC	Foundational concept practice. Clean inputs, guided structure, core syntax drilling.
INTERMEDIATE	Real-world complexity. Multiple concepts combined, edge cases, design decisions.
ADVANCED	Deep mastery. Complex algorithms, system design, performance analysis, creative problem solving.

Module-by-Module Topics Covered:

01	**Module 1: Introduction to C++ & Program Structure** Program anatomy, #include, main(), cout/cin, compilation process, IDE setup, comments, escape sequences, namespaces
02	**Module 2: Data Types, Variables & Operators** int, float, double, char, bool, auto, sizeof, type casting, arithmetic/relational/logical/bitwise operators, precedence, const, literals
03	**Module 3: Conditional Statements** if, else, else if, nested if, switch-case, ternary operator (?:), logical operators in conditions, dangling else, short-circuit evaluation
04	**Module 4: Loops & Iteration** while, do-while, for loops, nested loops, break, continue, infinite loops, loop counters, sentinel values, accumulator pattern
05	**Module 5: Arrays & Vectors** 1D arrays, 2D arrays, array initialization, array traversal, sorting, searching, std::vector, push_back, size(), iterators
06	**Module 6: Functions** Function declaration/definition, parameters (by value, by reference), return types, default parameters, function overloading, recursion, scope, inline
07	**Module 7: Pointers & Dynamic Memory** Pointer declaration, dereferencing, pointer arithmetic, pointers and arrays, pointers to functions, new/delete, dynamic arrays, nullptr, references vs pointers
08	**Module 8: Strings & String Operations** C-strings (char arrays), std::string class, string methods (length, substr, find, replace, append, compare), getline(), string streams, string conversions

01 Module 1: Introduction to C++ & Program Structure

Understanding the C++ build process, writing your first programs, using standard I/O, and learning the structure every C++ program follows.

TOPICS: Program anatomy, #include, main(), cout/cin, compilation process, IDE setup, comments, escape sequences, namespaces

BASIC

Basic Projects

Project 1: Personal Profile Card Generator

Write a program that uses cout with escape sequences (\n, \t, \\, \") to print a formatted profile card showing name, age, city, and a hobby. Practice endl vs \n, string literals, and comment styles (// and /* */).

KEY CONCEPTS PRACTICED:

• cout & insertion operator (<<) • Escape sequences (\n, \t, \") • Single-line & multi-line comments • Program structure: main() & return 0

EXPECTED OUTPUT: Formatted text box with aligned name, age, city, and hobby using tabs and newlines.

★ **BONUS CHALLENGE:** Add a decorative border using repeated characters like *, -, and | with precise alignment.

Project 2: Interactive Greeting Machine

Create a program that reads the user's first name and birth year using cin, computes their approximate age, and outputs a personalized greeting. Introduces cin >> for input, basic arithmetic, and the using namespace std directive.

KEY CONCEPTS PRACTICED:

• cin & extraction operator (>>) • using namespace std • Basic arithmetic in output • Variable declaration for input storage

EXPECTED OUTPUT: "Hello, Ahmed! You are approximately 22 years old. Welcome to C++!"

★ **BONUS CHALLENGE:** Handle the case where the user enters a future birth year by printing a humorous error message.

INTERMEDIATE

Intermediate Projects

Project 3: ASCII Art Scene Renderer

Build a program that renders a multi-line ASCII art scene (a house, tree, or landscape) using carefully formatted cout statements. Requires precise use of escape sequences, raw string literals (R"()"), and understanding how the compiler treats whitespace in output.

KEY CONCEPTS PRACTICED:

• Raw string literals R"(...)" • Multiple cout chaining • Whitespace control in output • Compilation: source -> object -> executable

EXPECTED OUTPUT: A 15-20 line ASCII art house with chimney, windows, door, path, and tree.

★ **BONUS CHALLENGE:** Let the user choose between 3 different scenes (house, rocket, animal) using a simple cin input before drawing.

Project 4: Multi-File Hello World with Makefile

Split a program into a header file (.h), an implementation file (.cpp), and a main file. Write a simple Makefile or build script. Understand the preprocessor (#include, #ifndef), separate compilation, and linking.

KEY CONCEPTS PRACTICED:

• Header files & #include guards • Separate compilation & linking • #ifndef / #define / #endif • Preprocessor directives overview

EXPECTED OUTPUT: Program compiles from 3 files and prints a formatted greeting assembled from functions across files.

★ **BONUS CHALLENGE:** Add a second module that provides a function returning the current date as a string, included via its own header.

ADVANCED Advanced Projects

Project 5: Build System Simulator

Create a program that simulates the C++ compilation pipeline: read a 'source file' (text from cin), show each phase (preprocessing, compilation, assembly, linking) with simulated output at each step. Deepens understanding of what happens between writing code and running an executable.

KEY CONCEPTS PRACTICED:

• C++ compilation pipeline stages • Preprocessor, compiler, assembler, linker • Object files and executables • Namespace design and usage

EXPECTED OUTPUT: Step-by-step display: [PREPROCESSOR] Expanding includes... [COMPILER] Generating assembly... [ASSEMBLER] Creating object file... [LINKER] Producing executable...

★ **BONUS CHALLENGE:** Detect and report simulated 'errors' (e.g., missing semicolons or unmatched braces in user input) at the correct phase.

Project 6: Interactive C++ Syntax Reference Tool

Build a mini reference system where the user types a keyword (cout, cin, include, main, namespace) and gets a formatted explanation with example code printed to the console. Uses structured output, escape sequences for showing code in output, and modular design.

KEY CONCEPTS PRACTICED:

• Structured program design • Complex string output with code examples • Input validation basics • Program as a learning tool

EXPECTED OUTPUT: User types 'cout' -> program prints definition, syntax, 3 examples with proper escaping of quotes and angle brackets.

★ **BONUS CHALLENGE:** Add a 'quiz' mode that asks the user syntax questions and checks their text answers.

02 Module 2: Data Types, Variables & Operators

Mastering C++ fundamental data types, variable declaration, initialization, type conversions, and all operator families with their precedence rules.

TOPICS: int, float, double, char, bool, auto, sizeof, type casting, arithmetic/relational/logical/bitwise operators, precedence, const, literals

BASIC Basic Projects

Project 1: Universal Unit Converter

Build a converter that handles temperature (C/F/K), distance (km/mi/m/ft), and weight (kg/lb/g/oz). Practice int vs double for precision, const for conversion factors, and formatted output with fixed decimal places using cout manipulators.

KEY CONCEPTS PRACTICED:

• int vs float vs double precision • const variable declarations • Arithmetic operators (+, -, *, /) • Type casting (static_cast)

EXPECTED OUTPUT: User enters 100 C -> "100.00 C = 212.00 F = 373.15 K"

★ **BONUS CHALLENGE:** Show the sizeof() each data type used and explain why double gives better precision than float for scientific conversions.

Project 2: Character Code Explorer

Create a program that takes a character input and displays its ASCII code, whether it's uppercase/lowercase/digit/special, and performs character arithmetic (e.g., shift by N positions). Teaches char as a numeric type and implicit type conversions.

KEY CONCEPTS PRACTICED:

• char type and ASCII values • Implicit int-char conversions • Relational operators (>=, <=, ==) • Bool type and logical expressions

EXPECTED OUTPUT: Input 'A' -> "ASCII: 65 | Type: Uppercase Letter | Shifted +3: D (68)"

★ **BONUS CHALLENGE:** Build a simple Caesar cipher that shifts all characters in a word by a user-specified amount, wrapping around Z->A.

INTERMEDIATE Intermediate Projects

Project 3: Bitwise Operations Visualizer

Read two integers and display the binary representation of each, then show the result of AND, OR, XOR, NOT, left-shift, and right-shift with both binary and decimal output. Makes abstract bit operations tangible and visual.

KEY CONCEPTS PRACTICED:

• Bitwise operators (&, |, ^, ~, <<, >>) • Binary number representation • Unsigned vs signed integers • Operator precedence rules

EXPECTED OUTPUT: A=12 (00001100), B=10 (00001010) -> AND=8 (00001000), OR=14 (00001110), XOR=6 (00000110)

★ **BONUS CHALLENGE:** Implement a function that counts the number of set bits (1s) in an integer using only bitwise operations.

Project 4: Financial Compound Interest Calculator

Calculate compound interest with monthly/quarterly/yearly compounding, comparing float vs double precision drift over 30 years. Use auto type deduction, demonstrate operator precedence with complex formulas, and show how floating-point errors accumulate.

KEY CONCEPTS PRACTICED:

• auto type deduction • Floating-point precision limits • Complex expression precedence • Assignment operators (+=, *=, etc.)

EXPECTED OUTPUT: Principal: $10,000 | Rate: 7% | 30 years -> Float result: $76,122.55 | Double result: $76,122.55 | Difference: $0.03

★ **BONUS CHALLENGE:** Add a side-by-side comparison table showing year-by-year growth and the growing float vs double divergence.

ADVANCED Advanced Projects

Project 5: Data Type Limits & Overflow Detector

Build a comprehensive tool that displays the range, size, and limits of every fundamental C++ type using and . Demonstrate integer overflow, underflow, and undefined behavior. Let the user input a number and determine the smallest type that can hold it.

KEY CONCEPTS PRACTICED:

• and constants • Integer overflow and undefined behavior • sizeof() for all types • Narrowing conversions and data loss

EXPECTED OUTPUT: Number 40000 -> "Fits in: unsigned short (0-65535), int, long, long long. Does NOT fit in: short (-32768 to 32767), char"

★ **BONUS CHALLENGE:** Simulate overflow by incrementing a short int past its max and showing the wrap-around behavior step by step.

Project 6: Expression Parser & Precedence Evaluator

Create a program that takes a simple arithmetic expression as a string (e.g., "3 + 4 * 2"), evaluates it respecting C++ operator precedence, and shows each step. Requires understanding all precedence levels and associativity rules deeply.

KEY CONCEPTS PRACTICED:

• Complete operator precedence table • Left-to-right vs right-to-left associativity • Parentheses and evaluation order • Ternary operator (?:) usage

EXPECTED OUTPUT: "3 + 4 * 2 - 1" -> Step 1: 4*2=8 -> Step 2: 3+8=11 -> Step 3: 11-1=10 -> Result: 10

★ **BONUS CHALLENGE:** Support parentheses, unary minus, and the modulo operator in the expression evaluator.

03 Module 3: Conditional Statements

Controlling program flow with conditional branches, mastering multi-way decisions, and understanding logical evaluation strategies.

TOPICS: if, else, else if, nested if, switch-case, ternary operator (?:), logical operators in conditions, dangling else, short-circuit evaluation

BASIC Basic Projects

Project 1: Student Grade Classification System

Read a numeric score (0-100) and classify it into letter grades (A+, A, B+, B, C, D, F) using if-else if chains. Handle invalid inputs, display remarks ("Excellent", "Good", "Needs Improvement"), and show pass/fail status.

KEY CONCEPTS PRACTICED:

• if / else if / else chains • Relational operators in conditions • Input validation with if • Logical AND (&&) for range checking

EXPECTED OUTPUT: Score: 87 -> "Grade: B+ | Status: PASS | Remark: Very Good | GPA Points: 3.5"

★ **BONUS CHALLENGE:** Add weighted categories (exam 60%, assignments 30%, participation 10%) before computing the final grade.

Project 2: Day Planner Decision Engine

Build a program that asks about today's conditions (weekday/weekend, weather, energy level) and suggests activities using nested if-else statements. Teaches nesting depth, logical operators, and combining multiple conditions.

KEY CONCEPTS PRACTICED:

• Nested if-else statements • Logical operators (&&, ||, !) • Combining multiple conditions • Boolean variables in conditions

EXPECTED OUTPUT: Weekend + Sunny + High Energy -> "Go hiking!" | Weekday + Rainy + Low Energy -> "Read a book at home"

★ **BONUS CHALLENGE:** Add a time-of-day factor (morning/afternoon/evening) creating a 3-level nesting structure with at least 12 unique outcomes.

INTERMEDIATE Intermediate Projects

Project 3: Scientific Calculator with Switch-Case

Build a calculator that reads two numbers and an operator (+, -, *, /, %, ^) using switch-case. Handle division by zero, integer vs floating-point division, and add scientific functions (sqrt, abs, power). Compare switch vs if-else efficiency.

KEY CONCEPTS PRACTICED:

• switch-case with char/int • break and fall-through behavior • default case for error handling • switch vs if-else comparison

EXPECTED OUTPUT: Enter: 15 / 4 -> "Integer division: 3 | Real division: 3.75 | Remainder: 3"

★ **BONUS CHALLENGE:** Add a memory feature (M+, M-, MR, MC) that stores and recalls values across operations using a variable.

Project 4: Leap Year & Calendar Validator

Validate a date (day/month/year) checking leap years, month lengths, and range validity using complex nested conditions. Then determine the day of the week using Zeller's formula. Showcases short-circuit evaluation and complex boolean logic.

KEY CONCEPTS PRACTICED:

• Short-circuit evaluation (&&, ||) • Complex nested conditionals • Ternary operator (?:) for concise logic • Dangling else problem and braces

EXPECTED OUTPUT: Date: 29/02/2024 -> "VALID | Leap Year: Yes | Day: Thursday | Days until year end: 306"

★ **BONUS CHALLENGE:** Calculate the number of days between two user-entered dates, handling all edge cases including century leap year rules.

ADVANCED Advanced Projects

Project 5: Text-Based Adventure Game Engine

Create a multi-room adventure game where every room presents 2-3 choices leading to different outcomes. Implement an inventory system (using variables/bools), health tracking, and at least 15 unique decision points. Demonstrates deeply nested conditionals and state management.

KEY CONCEPTS PRACTICED:

• Deep conditional nesting (4+ levels) • State management with bool variables • Complex multi-path decision trees • Combining all conditional constructs

EXPECTED OUTPUT: A 5-room dungeon with branching paths, item pickups, traps, a final boss encounter, and 4 possible endings.

★ **BONUS CHALLENGE:** Add a scoring system that tracks decisions and gives a different title (Coward, Explorer, Hero, Legend) based on the path taken.

Project 6: Income Tax Computation Engine

Implement a real-world progressive tax calculator with multiple brackets, deductions, filing status (single/married/head of household), and credits. Requires carefully ordered else-if chains, complex boolean conditions, and precise floating-point currency handling.

KEY CONCEPTS PRACTICED:

• Multi-bracket progressive logic • Enum-like constants for categories • Precise floating-point for currency • Complex condition ordering matters

EXPECTED OUTPUT: Income: $85,000 | Filing: Single -> Bracket breakdown, deductions applied, effective tax rate, take-home pay calculated.

★ **BONUS CHALLENGE:** Add comparison mode that shows the tax for all filing statuses side-by-side and recommends the optimal one.

04 Module 4: Loops & Iteration

Repeating actions with different loop constructs, controlling iterations with break/continue, and mastering nested loop patterns.

TOPICS: while, do-while, for loops, nested loops, break, continue, infinite loops, loop counters, sentinel values, accumulator pattern

BASIC Basic Projects

Project 1: Multiplication Table Generator

Generate formatted multiplication tables (1x1 through 12x12) using nested for loops with aligned columns. Let the user choose a specific table or the full grid. Practice loop counters, formatting with setw(), and nested iteration.

KEY CONCEPTS PRACTICED:

• for loop syntax and mechanics • Nested for loops (2 levels) • Loop counter variables • Output formatting with setw()

EXPECTED OUTPUT: A neatly aligned 12x12 grid with headers, or a single table like "7 x 1 = 7" through "7 x 12 = 84".

★ **BONUS CHALLENGE:** Add an option to generate tables in reverse order and highlight all perfect squares in the full grid.

Project 2: Number Guessing Game

Generate a random number (1-100) and let the user guess with "Too High" / "Too Low" feedback using a while loop. Track attempt count, enforce a maximum guess limit (10), and use do-while for replay. Teaches sentinel-controlled loops.

KEY CONCEPTS PRACTICED:

• while loop with conditions • do-while for menu replay • Loop termination conditions • Random number generation (srand, rand)

EXPECTED OUTPUT: "Guess #3: 42 -> Too Low! | Guess #4: 67 -> Too High! | Guess #5: 55 -> Correct in 5 attempts!"

★ **BONUS CHALLENGE:** Add difficulty levels (Easy: 1-50, Medium: 1-100, Hard: 1-1000) with different max attempt limits.

INTERMEDIATE Intermediate Projects

Project 3: Pattern Printing Workshop

Print 8 different patterns (right triangle, inverted triangle, pyramid, diamond, hollow rectangle, number pyramid, Floyd's triangle, Pascal-like pattern) using nested loops. Each pattern requires different loop logic and teaches how inner/outer loop relationships create shapes.

KEY CONCEPTS PRACTICED:

• Nested loops for 2D patterns • Space and character counting logic • break and continue in loops • Relationship between outer/inner counters

EXPECTED OUTPUT: 8 distinct patterns printed with user-chosen size N, each demonstrating a different nested loop technique.

★ **BONUS CHALLENGE:** Let the user input a custom character and generate a Christmas tree pattern with decorations at random positions.

Project 4: Prime Number Toolkit

Build a toolkit that: (1) checks if a number is prime, (2) lists all primes up to N using the Sieve of Eratosthenes concept with nested loops, (3) finds the prime factorization of a number, and (4) finds twin primes. Intensive loop practice with optimization.

KEY CONCEPTS PRACTICED:

• Optimized loop termination (sqrt) • Nested loops for sieve logic • continue to skip iterations • Accumulator and flag patterns

EXPECTED OUTPUT: N=50 -> Primes: 2,3,5,7,11,13... | Factor(60)=2x2x3x5 | Twin primes: (3,5),(5,7),(11,13)...

★ **BONUS CHALLENGE:** Implement a performance counter that shows how many iterations each method uses, comparing brute-force vs optimized.

ADVANCED Advanced Projects

Project 5: Console-Based Snake Game (Simplified)

Create a simplified snake game in the console using an infinite game loop with break conditions. The snake moves automatically, the user enters direction commands, and the game updates a text-based grid each frame. Combines infinite loops, nested loops for grid rendering, and break/continue for game logic.

KEY CONCEPTS PRACTICED:

• Infinite loops with break conditions • Game loop architecture • Nested loops for grid rendering • Complex loop control flow

EXPECTED OUTPUT: A text-based grid updating each turn showing a snake (#) moving toward food (*) with a growing score.

★ **BONUS CHALLENGE:** Add walls (boundaries that kill the snake) and increasing speed (shorter delay) as the score grows.

Project 6: Numerical Methods Calculator

Implement iterative numerical algorithms: (1) Newton-Raphson square root approximation, (2) computing Pi using Leibniz series, (3) computing e using factorial series, (4) finding GCD via Euclidean algorithm. Each uses loops with convergence-based termination rather than fixed counts.

KEY CONCEPTS PRACTICED:

• Convergence-based loop termination • Accumulator pattern for series • Nested loops for iterative methods • Epsilon comparison for doubles

EXPECTED OUTPUT: sqrt(2) converges to 1.41421356... in 6 iterations | Pi after 1M terms: 3.14159165... | e: 2.71828182...

★ **BONUS CHALLENGE:** Add a visual convergence graph (text-based) showing how each approximation approaches the true value per iteration.

05 Module 5: Arrays & Vectors

Storing and manipulating collections of data using fixed-size arrays and dynamic vectors, with sorting and searching algorithms.

TOPICS: 1D arrays, 2D arrays, array initialization, array traversal, sorting, searching, std::vector, push_back, size(), iterators

BASIC Basic Projects

Project 1: Student Marks Analyzer

Store marks for N students in an array, then compute average, highest, lowest, count above/below average, and display a simple histogram. Practice array declaration, initialization, traversal with for loops, and passing arrays to simple operations.

KEY CONCEPTS PRACTICED:

• Array declaration & initialization • Array traversal with for loops • Finding min/max in arrays • Computing statistics from array data

EXPECTED OUTPUT: Marks: [78,92,65,88,45,91] -> Avg: 76.5 | Max: 92 | Min: 45 | Above avg: 4 students | Histogram bars.

★ **BONUS CHALLENGE:** Sort the marks in descending order using bubble sort and display the ranking with student numbers.

Project 2: Dynamic Shopping List Manager

Build a shopping list using std::vector with options to add, remove, display, and search for items. Introduces vector basics (push_back, pop_back, size, empty, at) and why vectors are preferred over raw arrays for dynamic data.

KEY CONCEPTS PRACTICED:

• std::vector declaration & usage • push_back(), pop_back(), size() • Vector traversal (index & range-for) • Vector vs array comparison

EXPECTED OUTPUT: Menu: [1]Add [2]Remove [3]Display [4]Search [5]Clear [6]Exit -> Manages a dynamic item list.

★ **BONUS CHALLENGE:** Add quantity tracking (vector of structs or parallel vectors) and a total item count display.

INTERMEDIATE Intermediate Projects

Project 3: Tic-Tac-Toe Game with 2D Array

Implement a two-player Tic-Tac-Toe game using a 3x3 2D char array for the board. Check win conditions (rows, columns, diagonals), validate moves, detect draws, and display the formatted board after each turn. Core 2D array manipulation project.

KEY CONCEPTS PRACTICED:

• 2D array declaration & access • Row/column/diagonal traversal • 2D array as game board • Multi-dimensional array passing

EXPECTED OUTPUT: A playable 3x3 board with X/O placement, win detection, draw detection, and replay option.

★ **BONUS CHALLENGE:** Add a simple AI opponent that blocks winning moves and takes the center when available.

Project 4: Sorting Algorithm Visualizer

Implement Bubble Sort, Selection Sort, and Insertion Sort on integer arrays. After each swap or pass, display the array state with the elements being compared highlighted. Count and compare total comparisons and swaps across algorithms.

KEY CONCEPTS PRACTICED:

• Sorting algorithms on arrays • Nested loops for sorting • Swap operations and temp variables • Algorithm comparison metrics

EXPECTED OUTPUT: Array [64,34,25,12,22,11,90] -> Step-by-step sorting with [brackets] around compared elements, swap count per algorithm.

★ **BONUS CHALLENGE:** Generate random arrays of size 100/1000/10000 and create a comparison table of swap counts and comparisons for all three algorithms.

ADVANCED Advanced Projects

Project 5: Matrix Operations Library

Build a complete matrix math library using 2D arrays: addition, subtraction, scalar multiplication, matrix multiplication, transpose, and determinant (for 2x2 and 3x3). Include input validation for dimension compatibility and formatted matrix output.

KEY CONCEPTS PRACTICED:

• 2D array for matrix storage • Triple-nested loops for multiplication • Array size passing to functions • Mathematical operations on 2D data

EXPECTED OUTPUT: Matrix A (3x2) * Matrix B (2x4) = Result (3x4), with formatted aligned output for all operations.

★ **BONUS CHALLENGE:** Add matrix inverse computation for 3x3 matrices using cofactor expansion and the adjugate method.

Project 6: Student Database with Vector of Records

Create a student management system using vectors with parallel data tracking (names, IDs, grades across subjects). Implement search (linear & binary on sorted data), multi-key sorting, filtering, and statistical reporting across the dataset.

KEY CONCEPTS PRACTICED:

• Vectors of complex data • Binary search on sorted vectors • Multi-criteria sorting • Iterator-based operations

EXPECTED OUTPUT: Full CRUD on student records: Add, Search by ID/name, Sort by GPA/name, Filter by grade range, Generate class statistics report.

★ **BONUS CHALLENGE:** Export the database to a formatted text file and import it back on program restart (basic file persistence).

06 Module 6: Functions

Decomposing programs into reusable, modular functions with proper parameter passing, overloading, and recursive problem solving.

TOPICS: Function declaration/definition, parameters (by value, by reference), return types, default parameters, function overloading, recursion, scope, inline

BASIC

Basic Projects

Project 1: Math Utilities Function Library

Create a library of 10+ math functions: factorial, power, absolute value, min/max of 2 and 3 numbers, isEven, isPrime, GCD, LCM, and fibonacci. Practice function declaration, definition, calling, and return values. Each function is called from a menu-driven main().

KEY CONCEPTS PRACTICED:

• Function declaration & definition • Parameters and return values • Function calls and control flow • Scope: local vs global variables

EXPECTED OUTPUT: Menu-driven: User picks a function, enters inputs, gets result. E.g., GCD(48,18) = 6, 8! = 40320.

★ **BONUS CHALLENGE:** Add a 'chain' mode where the output of one function feeds as input to another (e.g., factorial(GCD(a,b))).

Project 2: Temperature & Currency Converter Suite

Build converter functions for temperature (C/F/K), currency (USD/EUR/GBP/PKR with rates), and distance (metric/imperial). Practice pass-by-value, meaningful return values, and function overloading (same name, different parameter types/counts).

KEY CONCEPTS PRACTICED:

• Pass-by-value semantics • Function overloading (same name, diff params) • Default parameter values • Multiple return via reference parameters

EXPECTED OUTPUT: convert(100, 'C', 'F') = 212.0 | convert(50, "USD", "EUR") = 46.25

★ **BONUS CHALLENGE:** Overload a convert() function to work with both single values and arrays of values.

INTERMEDIATE

Intermediate Projects

Project 3: Array Processing with Pass-by-Reference

Write functions that operate on arrays passed by reference: fillRandom(), sortArray(), reverseArray(), findMedian(), removeElement(), insertSorted(). Demonstrate how arrays are always passed by pointer, and use reference parameters for returning multiple values (e.g., min and max simultaneously).

KEY CONCEPTS PRACTICED:

• Pass-by-reference (&) vs by-value • Array passing to functions • Reference parameters for multiple returns • const reference for read-only access

EXPECTED OUTPUT: Array processed through pipeline: fill -> display -> sort -> find median -> remove duplicates -> display final.

★ **BONUS CHALLENGE:** Implement a generic stats() function that returns mean, median, mode, std deviation, and range via reference parameters in one call.

Project 4: Recursive Art: Fractals in Text

Implement recursive functions to draw text-based fractals: Sierpinski Triangle, recursive tree branching, and recursive star patterns. Each function calls itself with reduced size/depth. Visualizes the call stack and recursion depth clearly.

KEY CONCEPTS PRACTICED:

• Recursion: base case & recursive case • Call stack visualization • Recursion depth and stack overflow • Recursive vs iterative comparison

EXPECTED OUTPUT: Sierpinski triangle of depth 4 drawn with characters, recursive tree with branching '*' patterns.

★ **BONUS CHALLENGE:** Add a recursion depth counter that prints the current depth at each call, showing exactly how the call stack grows and shrinks.

ADVANCED Advanced Projects

Project 5: Recursive Puzzle Solver (Towers of Hanoi & N-Queens)

Implement two classic recursive algorithms: Towers of Hanoi (move N disks between 3 pegs with step-by-step display) and a simplified N-Queens solver (place N queens on NxN board with no attacks). Shows recursion for complex problem decomposition and backtracking.

KEY CONCEPTS PRACTICED:

• Complex recursion with backtracking • Recursive decomposition of problems • Tail recursion concept • Function pointers (intro) for callbacks

EXPECTED OUTPUT: Hanoi(4): 15 moves listed step by step with peg states | N-Queens(8): board layout with Q placements.

★ **BONUS CHALLENGE:** Count total recursive calls for Hanoi(N) and Queens(N), comparing against theoretical complexity (2^n - 1 for Hanoi).

Project 6: Mini Testing Framework

Build a lightweight testing framework with assert-like functions: assertEquals(), assertTrue(), assertArrayEquals(), and a test runner that calls test functions, counts passes/fails, and reports results. Uses function overloading extensively and demonstrates functions as building blocks of software quality.

KEY CONCEPTS PRACTICED:

• Function overloading for type-specific asserts • Inline functions for performance • Functions calling other functions • Software engineering with functions

EXPECTED OUTPUT: Running 15 tests... [PASS] testAdd [PASS] testSort [FAIL] testSearch (expected 5, got -1) | Results: 14/15 passed.

★ **BONUS CHALLENGE:** Add a timing feature that measures each test function's execution time in microseconds using .

07 Module 7: Pointers & Dynamic Memory

Understanding memory addresses, pointer operations, dynamic memory allocation, and the relationship between pointers, arrays, and functions.

TOPICS: Pointer declaration, dereferencing, pointer arithmetic, pointers and arrays, pointers to functions, new/delete, dynamic arrays, nullptr, references vs pointers

BASIC Basic Projects

Project 1: Pointer Playground & Visualizer

Create a program that declares variables of different types, creates pointers to them, and displays variable values, addresses (&), and dereferenced pointer values (*) in a formatted table. Visually demonstrate that a pointer 'holds an address' and dereferencing 'follows the address to the value'.

KEY CONCEPTS PRACTICED:

• Pointer declaration (int* p) • Address-of operator (&) • Dereference operator (*) • nullptr and null pointer safety

EXPECTED OUTPUT: | Variable | Value | Address | Pointer | Points To | *Ptr Value |\n| x | 42 | 0x7ffd... | px | 0x7ffd... | 42 |

★ **BONUS CHALLENGE:** Show pointer-to-pointer (int**) with a three-level chain: variable -> pointer -> pointer-to-pointer, all with addresses.

Project 2: Swap Functions: Value vs Pointer vs Reference

Implement three versions of a swap function: by value (fails), by pointer (works), and by reference (works). Display variable values before and after each call to prove which methods actually modify the originals. The definitive lesson on why pointers and references matter.

KEY CONCEPTS PRACTICED:

• Pass-by-pointer (* and &) • Pointer parameters in functions • References vs pointers comparison • Why pass-by-value can't swap

EXPECTED OUTPUT: Before: a=5, b=10 | After swapByValue: a=5, b=10 (unchanged!) | After swapByPtr: a=10, b=5 (swapped!)

★ **BONUS CHALLENGE:** Extend to a rotateThree() function that cyclically rotates three values using only pointer operations.

INTERMEDIATE Intermediate Projects

Project 3: Dynamic Array Manager

Build a resizable array from scratch using new/delete. Implement addElement (with reallocation when full, doubling capacity), removeElement, getElement, and display. Track size vs capacity. This is essentially building a simplified std::vector to understand what it does internally.

KEY CONCEPTS PRACTICED:

• new and delete operators • new[] and delete[] for arrays • Dynamic memory allocation strategy • Memory leaks and proper deallocation

EXPECTED OUTPUT: DynArray: size=5, capacity=8 | Add(99): size=6, capacity=8 | Add 3 more: size=9, capacity=16 (doubled!)

★ **BONUS CHALLENGE:** Add a shrinkToFit() function and implement a copy function that deep-copies one dynamic array to another.

Project 4: Pointer-Based String Operations

Implement C-string functions using only pointer arithmetic (no index []): myStrlen, myStrcpy, myStrcat, myStrcmp, myStrrev, and myStrFind. Each function uses pointer incrementing (p++) to traverse characters. Deepens understanding of pointers and arrays being interchangeable.

KEY CONCEPTS PRACTICED:

• Pointer arithmetic (p++, p+n) • Pointer and array equivalence • C-string manipulation via pointers • Pointer comparison and subtraction

EXPECTED OUTPUT: myStrlen("Hello")=5 | myStrcat("Hello"," World")="Hello World" | myStrrev("abc")="cba"

★ **BONUS CHALLENGE:** Implement myStrtok() that splits a string by a delimiter using only pointer manipulation, returning tokens one at a time.

ADVANCED Advanced Projects

Project 5: Memory Pool Allocator Simulation

Simulate a simple memory pool: allocate a large block with new, then manage sub-allocations from it (allocate, deallocate, display memory map). Show fragmentation, track used/free blocks, and visualize the memory layout as a text-based map. Teaches how memory management works under the hood.

KEY CONCEPTS PRACTICED:

• Large block allocation with new • Pointer offset calculations • Memory fragmentation visualization • Manual memory management challenges

EXPECTED OUTPUT: [USED:24B][FREE:8B][USED:16B][FREE:48B][USED:32B] | Total: 128B | Used: 72B | Free: 56B | Fragments: 2

★ **BONUS CHALLENGE:** Implement a defragmentation routine that compacts all used blocks together and merges free blocks.

Project 6: Linked List from Scratch with Pointers

Build a singly linked list using structs and pointers: insert (front, back, at position), delete (by value, by position), search, reverse, display, and count. The foundational data structure project that makes pointers click for most students.

KEY CONCEPTS PRACTICED:

• Struct with pointer member (Node*) • Dynamic node creation (new Node) • Traversal via pointer chaining • Proper deletion to prevent leaks

EXPECTED OUTPUT: List: 10 -> 20 -> 30 -> 40 | InsertAt(2,25): 10 -> 20 -> 25 -> 30 -> 40 | Reverse: 40 -> 30 -> 25 -> 20 -> 10

★ **BONUS CHALLENGE:** Implement a sorted insert that maintains ascending order and a merge function that merges two sorted lists into one.

08 Module 8: Strings & String Operations

Working with text data using both C-style character arrays and the modern std::string class, with comprehensive string manipulation techniques.

TOPICS: C-strings (char arrays), std::string class, string methods (length, substr, find, replace, append, compare), getline(), string streams, string conversions

BASIC Basic Projects

Project 1: Word Counter & Text Statistics

Read a sentence with getline() and compute: character count, word count, sentence count, average word length, longest/shortest word, and vowel/consonant ratio. Practice std::string methods (length, at, find) and character-by-character traversal.

KEY CONCEPTS PRACTICED:

• std::string declaration & input • getline() for full-line input • String length() and at() • Character classification (isalpha, isdigit)

EXPECTED OUTPUT: "Hello World CPP" -> Chars: 15 | Words: 3 | Avg word length: 4.3 | Longest: Hello (5) | Vowels: 4

★ **BONUS CHALLENGE:** Read multi-line input (terminated by an empty line) and produce full paragraph statistics including sentence count.

Project 2: Password Strength Validator

Check a password against rules: minimum 8 characters, at least one uppercase, one lowercase, one digit, one special character. Use std::string methods and character classification functions. Display which rules pass/fail and give a strength rating (Weak/Medium/Strong/Very Strong).

KEY CONCEPTS PRACTICED:

• String traversal and checking • String comparison operators • substr() for partial extraction • Character testing (isupper, islower, isdigit)

EXPECTED OUTPUT: Password: "MyP@ss1" -> [PASS] Length >= 8: NO (7) | [PASS] Uppercase: YES | [PASS] Special: YES | Rating: MEDIUM

★ **BONUS CHALLENGE:** Add a common password dictionary check (hardcoded list of 20 common passwords) and a similarity checker.

INTERMEDIATE Intermediate Projects

Project 3: Text Encryption/Decryption Tool

Implement three ciphers: Caesar (shift), Vigenere (key-based shift), and a simple substitution cipher. Each operates on std::string using find(), replace(), substr(), and character manipulation. User picks cipher, enters text and key, gets encrypted/decrypted output.

KEY CONCEPTS PRACTICED:

• String manipulation with substr() • find() and replace() methods • String concatenation and append() • String iteration with modification

EXPECTED OUTPUT: Caesar("HELLO", shift=3) = "KHOOR" | Vigenere("HELLO", key="KEY") = "RIJVS" | Decrypt reverses each.

★ **BONUS CHALLENGE:** Add a frequency analysis tool that attempts to break a Caesar cipher by analyzing letter distribution.

Project 4: CSV Line Parser with String Streams

Read CSV-formatted lines and parse them into individual fields using stringstream and getline with a delimiter. Handle quoted fields, missing values, and different delimiters (comma, tab, pipe). Store parsed data in vectors and display as a formatted table.

KEY CONCEPTS PRACTICED:

• std::stringstream for parsing • getline() with custom delimiter • String-to-number conversion (stoi, stod) • String stream extraction (>>)

EXPECTED OUTPUT: Input: "Alice,25,3.8,CS" -> | Name: Alice | Age: 25 | GPA: 3.8 | Major: CS | (formatted table)

★ **BONUS CHALLENGE:** Parse a multi-line CSV input and compute column-wise statistics (average age, GPA distribution, major counts).

ADVANCED Advanced Projects

Project 5: Regex-Free Text Search Engine

Build a text search tool supporting: exact match, case-insensitive search, wildcard matching (* and ?), and a simplified pattern matching (find all words starting/ending with a pattern). Implement all matching logic using only std::string methods (find, substr, compare) without regex.

KEY CONCEPTS PRACTICED:

• Complex find() with position tracking • String comparison (compare, ==) • npos and search result handling • Building algorithms from string primitives

EXPECTED OUTPUT: Text: 'The quick brown fox' | Search 'qu*ck' -> Found at position 4 | '*own' -> 'brown' at position 10

★ **BONUS CHALLENGE:** Add a replace-all function and a context display (show 10 characters before/after each match, like grep).

Project 6: Markdown-to-Plain-Text Converter

Parse a subset of Markdown syntax and convert to formatted plain text: # headings to UPPERCASE, **bold** to [text], *italic* to _text_, - lists to numbered lists, and `code` to . Requires complex string scanning, nested find() calls, and careful substr() extraction.

KEY CONCEPTS PRACTICED:

• Multi-pass string transformation • Nested find() for paired markers • String insertion and erasure • Complex string state machine logic

EXPECTED OUTPUT: Input: '# Title\n**Bold text** and *italic*' -> Output: 'TITLE\n======\n[Bold text] and _italic_'

★ **BONUS CHALLENGE:** Add support for [links](url) converting to 'text (url)' and handle nested formatting like ***bold italic***.

ALL 48 PROJECTS — QUICK REFERENCE

Complete index of all projects organized by module. Each project is unique and targets specific C++ Essentials 1 concepts from the Cisco CPA curriculum.

#	MODULE	LEVEL	PROJECT TITLE
1	Introduction to C++ & Pro	Basic	Personal Profile Card Generator
2	Introduction to C++ & Pro	Basic	Interactive Greeting Machine
3	Introduction to C++ & Pro	Intermediate	ASCII Art Scene Renderer
4	Introduction to C++ & Pro	Intermediate	Multi-File Hello World with Makefile
5	Introduction to C++ & Pro	Advanced	Build System Simulator
6	Introduction to C++ & Pro	Advanced	Interactive C++ Syntax Reference Tool
7	Data Types, Variables & O	Basic	Universal Unit Converter
8	Data Types, Variables & O	Basic	Character Code Explorer
9	Data Types, Variables & O	Intermediate	Bitwise Operations Visualizer
10	Data Types, Variables & O	Intermediate	Financial Compound Interest Calculator
11	Data Types, Variables & O	Advanced	Data Type Limits & Overflow Detector
12	Data Types, Variables & O	Advanced	Expression Parser & Precedence Evaluator
13	Conditional Statements	Basic	Student Grade Classification System
14	Conditional Statements	Basic	Day Planner Decision Engine
15	Conditional Statements	Intermediate	Scientific Calculator with Switch-Case
16	Conditional Statements	Intermediate	Leap Year & Calendar Validator
17	Conditional Statements	Advanced	Text-Based Adventure Game Engine
18	Conditional Statements	Advanced	Income Tax Computation Engine
19	Loops & Iteration	Basic	Multiplication Table Generator
20	Loops & Iteration	Basic	Number Guessing Game
21	Loops & Iteration	Intermediate	Pattern Printing Workshop
22	Loops & Iteration	Intermediate	Prime Number Toolkit
23	Loops & Iteration	Advanced	Console-Based Snake Game (Simplified)
24	Loops & Iteration	Advanced	Numerical Methods Calculator
25	Arrays & Vectors	Basic	Student Marks Analyzer
26	Arrays & Vectors	Basic	Dynamic Shopping List Manager
27	Arrays & Vectors	Intermediate	Tic-Tac-Toe Game with 2D Array
28	Arrays & Vectors	Intermediate	Sorting Algorithm Visualizer
29	Arrays & Vectors	Advanced	Matrix Operations Library
30	Arrays & Vectors	Advanced	Student Database with Vector of Records
31	Functions	Basic	Math Utilities Function Library
32	Functions	Basic	Temperature & Currency Converter Suite

#	MODULE	LEVEL	PROJECT TITLE
33	Functions	Intermediate	Array Processing with Pass-by-Reference
34	Functions	Intermediate	Recursive Art: Fractals in Text
35	Functions	Advanced	Recursive Puzzle Solver (Towers of Hanoi & N-Queens)
36	Functions	Advanced	Mini Testing Framework
37	Pointers & Dynamic Memory	Basic	Pointer Playground & Visualizer
38	Pointers & Dynamic Memory	Basic	Swap Functions: Value vs Pointer vs Reference
39	Pointers & Dynamic Memory	Intermediate	Dynamic Array Manager
40	Pointers & Dynamic Memory	Intermediate	Pointer-Based String Operations
41	Pointers & Dynamic Memory	Advanced	Memory Pool Allocator Simulation
42	Pointers & Dynamic Memory	Advanced	Linked List from Scratch with Pointers
43	Strings & String Operatio	Basic	Word Counter & Text Statistics
44	Strings & String Operatio	Basic	Password Strength Validator
45	Strings & String Operatio	Intermediate	Text Encryption/Decryption Tool
46	Strings & String Operatio	Intermediate	CSV Line Parser with String Streams
47	Strings & String Operatio	Advanced	Regex-Free Text Search Engine
48	Strings & String Operatio	Advanced	Markdown-to-Plain-Text Converter

RECOMMENDED TOOLS & SETUP

IDEs & Editors
Code::Blocks (recommended by Cisco), Visual Studio Code with C/C++ extension, CLion, Dev-C++, or any text editor with a terminal.

Compilers
GCC/G++ (Linux/Mac/MinGW), MSVC (Windows Visual Studio), Clang. Use C++17 standard: g++ -std=c++17 -Wall -o program main.cpp

Online Compilers
Replit, OnlineGDB, Compiler Explorer (godbolt.org), Programiz C++ compiler — great for quick testing without local setup.

Debugging
GDB (command line), Visual Studio Debugger, Code::Blocks built-in debugger. Learn breakpoints, step-through, and variable watches early.

Version Control
Git + GitHub/GitLab. Create a repository for your projects. Commit after each working feature. This builds your portfolio.

Reference
cppreference.com (comprehensive), cplusplus.com (beginner-friendly), Cisco NetAcad course materials, and this project guide.